# SenFEED: Dynamic Decentralized Oracle Services for Accurate and Real-Time Sensor Data

Hao Zeng[†], Helei Cui[†*], Cong Wang[‡], Bo Zhang[†], Zhiwen Yu[†§], and Bin Guo[†]
[†]School of Computer Science, Northwestern Polytechnical University, Xi'an, China
[‡]Department of Computer Science, City University of Hong Kong, Hong Kong, China
[§]College of Computer Science and Technology, Harbin Engineering University, Harbin, China

IEEE INFOCOM 2025 - IEEE Conference on Computer Communications | 979-8-3315-4305-1/25/$31.00 ©2025 IEEE | DOI: 10.1109/INFOCOM55648.2025.11044714

***Abstract*—Smart contracts, as self-executing programs on blockchains, have been applied in many Internet of Things (IoT) systems to achieve decentralized, fair, and auditable transaction services. However, since smart contracts cannot natively fetch real-world data, this limitation hinders the full economic potential of smart contracts in IoT. A promising solution is to leverage oracle services (also known as data feeds), which enable smart contracts to access and interact with external data sources. Nonetheless, these services must contend with the streaming nature of sensor data in IoT and the dynamic joining or leaving of service nodes in the blockchain. To address these challenges, we propose dynamic decentralized oracle services called SenFEED. Specifically, we integrate iterative-based and incremental-based truth discovery algorithms to achieve accurate and real-time truth inference on IoT streaming data. Furthermore, to address the challenge of reaching consensus in a decentralized setting, we design a new Byzantine fault-tolerant protocol that allows service nodes to join or leave without requiring a system reset. Theoretical analysis and extensive experiments confirm the effectiveness of SenFEED, demonstrating significant improvements in scenarios with a larger number of nodes and robust support for data streams in IoT systems.**

***Index Terms*—Internet of Things, blockchain oracle services, truth discovery, data quality.**

## I. Introduction

The Internet of Things (IoT) is being engaged in more and more areas, which interconnects diverse physical devices with sensors through the network for seamless communication and data exchange [1]–[3]. These sensors range from smart home appliances and industrial machines to wearable gadgets. However, the swift expansion of IoT has raised significant concerns regarding mutual distrust among sensors [4], privacy intrusions [5], and security vulnerabilities [6]. In response to these concerns, blockchain offers a decentralized framework with secure smart contract development to automate and enforce agreements among distrusted sensors [7], [8]. The risks of privacy intrusions and security vulnerabilities are mitigated through the immutable code, access controls, and security audits in smart contracts [4], [9].

To unleash the full potential of IoT-blockchain systems and facilitate innovative IoT applications, it is important for smart contracts to interact with real-world sensors seamlessly. For example, agricultural insurance smart contracts [10] collect weather data from field sensors to automatically disburse compensation for crop damage caused by adverse weather; logistics smart contracts [11] utilize IoT trackers on shipments to monitor their location for the automatic release of payments upon successful delivery. Although smart contracts offer numerous benefits for IoT applications, they can't proactively interact with real-world sensors [12].

To achieve this, third-party services namely *data feeds* (*a.k.a., oracles*) are applied to supplying real-world data to smart contracts [12], [13]. In this way, most existing studies further consider the decentralized service model to prevent a single point of failure [14]–[16]. Briefly, they utilize a special on-chain contract to aggregate data from multiple independent nodes (that are dedicated to *oracle services*) before transmitting the data to the target contracts. However, the data sources may provide low-quality or Byzantine data, so-called Byzantine sources, and this will lead to flawed smart contract results [16]. Meanwhile, the service nodes may manipulate data from the sources, so-called Byzantine nodes, and this will compromise the consistency and deterministic results of smart contracts [17]. These studies about the oracle services primarily concern data authentication [17]–[20] and data validation [14], [16], [21]–[23]. The former transports untampered data from authenticated websites to smart contracts via the trusted execution environment (TEE) [18], TLSNotary proof [24] or zero-knowledge proof [20]. The latter identifies the most accurate data from potentially conflicting sources via voting mechanisms [21] or truth discovery mechanisms [16].

In this trend, we desire to extend decentralized oracle services to the above-mentioned IoT-blockchain systems. The envisioned new oracle services, namely SenFEED, enable smart contracts to proactively access accurate and real-time data from real-world sensors of IoT devices. In brief, multiple service nodes independently verify the authentication of data from real-world sensors, identify the most accurate data by truth discovery mechanisms, achieve consensus on final data, and transmit it to smart contracts, ensuring the authentication and validation of the input data for automated execution programs in smart contracts.

Although SenFEED looks promising, there are still some challenges when implementing it for IoT-blockchain systems [25]. First, numerous uneven-quality IoT sensors generate streaming data (large volume data generated continuously and incrementally) of varying qualities, while existing truth discovery mechanisms could not satisfy both accuracy and real-time requirements [26], [27]. Second, the service nodes

*Corresponding author, email: chl@nwpu.edu.cn

Authorized licensed use limited to: BEIJING UNIVERSITY OF POST AND TELECOM. Downloaded on January 26,2026 at 06:08:16 UTC from IEEE Xplore. Restrictions apply.

may dynamically join or leave the network, while existing (static) Byzantine fault-tolerant (BFT) consensus [16], [17] may suffer *liveness* problems under the partially synchronous Byzantine-ridden situation (more discussion in Section III-B).

For the first challenge, we propose the Composite Truth Discovery (CompositeTD) algorithm for accurate real-time truth inference on IoT streaming data. It combines the strengths of iterative-based and incremental-based truth discovery to balance accuracy and efficiency. Normally, CompositeTD leverages maximum a posteriori (MAP) estimation to derive objects' truths from the arriving data. The incremental-based approach is efficient but may be inaccurate when sources are suddenly under Byzantine control (such as damaged sensors). By analyzing the unit error and cumulative error, CompositeTD can detect the Byzantine control on sources. It then alternates objects' truths estimation and sources' weights estimation until convergence to obtain accurate results.

For the second challenge, we develop the Dynamic Partially Synchronous BFT (DPS-BFT) protocol that permits service nodes to join or leave without compromising performance under the partially synchronous Byzantine-ridden network. Specifically, we design the dynamic decentralized oracle model for service nodes. Then we transform the static (partially synchronous) BFT protocol into a dynamic form, called DPS-BFT protocol, consisting of the network status discovery mechanism and the reliable view-change mechanism. The former mechanism forces nodes to catch up with the latest network status and the latter enables nodes to conduct the view-change when the leader node is manipulated or time out.

In summary, the contributions of this paper are as follows:
• We develop dynamic decentralized oracle services SenFEED, that enables smart contracts to access accurate and real-time sensor data proactively. SenFEED builds on top of our proposed CompositeTD algorithm for accurate real-time truth inference and DPS-BFT protocol for handling liveness problems arising as service nodes join or leave the network.
• We provide theoretical analysis to prove that CompositeTD algorithm meets both accuracy and real-time requirements in IoT scenarios. We also prove that DPS-BFT protocol is resistant to dynamic manipulation of service nodes and sources by Byzantine adversaries, and solves liveness problems.
• We conduct experiments to show that SenFEED permits service nodes to join or leave the network without compromising performance. Moreover, it is more accurate than Chainlink [14] and supports $4.8\times$ more nodes than DecenTruth [16] with lower latency and higher throughput.

## II. Problem Statement

### A. Dynamic Decentralized Oracle Model

For smart contracts to proactively fetch data from IoT sensors in almost real network environments, we formulate a dynamic decentralized oracle model for the IoT-blockchain system. We define a finite set of service nodes $\mathcal{U} = \{u_1, u_2, ...\}$ as the service node universe. Hereafter, we will refer to service nodes simply as nodes. At any given timestamp, we denote the *network status* as $\mathcal{U}_c$ which represents the set of active nodes (a subset of $\mathcal{U}$) in the network. The network status is indexed by $c$, with the initial status being $c = 0$. In the dynamic partially synchronous network, each node maintains its own network status, which may not be promptly synchronized with others. Thus, their views of the current network status $\mathcal{U}_c$ may differ. We further clarify the concept of the network status:

**Definition 1.** *The current network status of $u$ is $\mathcal{U}_c$, if the node $u$ has updated the network status to $\mathcal{U}_c$ but has not updated the network status to $\mathcal{U}_{c'}$ where $c' > c$.*

**Definition 2.** *The current network status of the system is $\mathcal{U}_c$, if any correct node has updated the network status to $\mathcal{U}_c$ and no correct node has updated it to $\mathcal{U}_{c'}$ where $c' > c$.*

To formulate the dynamic decentralized oracle model, we transform the static partially synchronous BFT consensus [28] into the dynamic partially synchronous BFT consensus. We assume that the number of nodes joining or leaving the network is finite, an unknown upper bound on message latency exists, and messages among nodes are eventually delivered within the bounded latency. The dynamic partially synchronous BFT for our oracle model has two crucial components, the view [28] and the network status. The former represents a configuration where a specific leader node is responsible for managing the consensus process, and the latter represents the set of active nodes within a period of time. Nodes are responsible for transporting real-time streaming data from real-world sensors to smart contracts. Specifically, given the set of IoT objects $\mathcal{O}$ and data sources $\mathcal{S}$ (IoT sensors), each source $s$ can independently monitor multiple IoT objects in real-time. $v_{s,o}^t$ represents the data generated by the source $s \in \mathcal{S}$ about the object $o$ at timestamp $t$. At any given timestamp, an object $o$ may be monitored by some sources (denoted as $\mathcal{S}_o \subseteq \mathcal{S}$). Nodes in $\mathcal{U}_c$ achieve consensus on objects' truths (*e.g.*, temperature and humidity) at timestamp $t$ in real-time. The authenticity and integrity of messages can be further guaranteed through TLS communication or other tools [18], [20].

### B. System Overview

SenFEED is a concrete instantiation of the dynamic decentralized oracle model in the partially synchronous network, which allows smart contracts to proactively access sensor data. The workflow is shown in Fig. 1, including two major entities: sources (*i.e.*, IoT devices with multiple independent sensors) and nodes. Sources independently and constantly monitor IoT objects to generate real-time streaming sensor data and nodes transport the data from sensors to smart contracts. Formally, each node $u_i$ can access different sources (denoted as $\mathcal{S}_{u_i} \subseteq \mathcal{S}$) and obtain data $\mathcal{V}_{u_i}^t = \{\mathbb{1}_{s,o} v_{s,o}^t\}_{s \in \mathcal{S}_u, o \in \mathcal{O}}$, where $\mathbb{1}_{s,o} = 1$ denotes that $s$ provides sensor data about object $o$ ($0$ otherwise). In other words, each source $s$ can provide sensor data about some IoT objects at timestamp $t$ in the streaming format (a continuous flow of data that is generated and processed in real-time).

Since each node collects different sensor data, data aggregation is required. For efficiency, we consider leader-centric

Authorized licensed use limited to: BEIJING UNIVERSITY OF POST AND TELECOM. Downloaded on January 26,2026 at 06:08:16 UTC from IEEE Xplore. Restrictions apply.

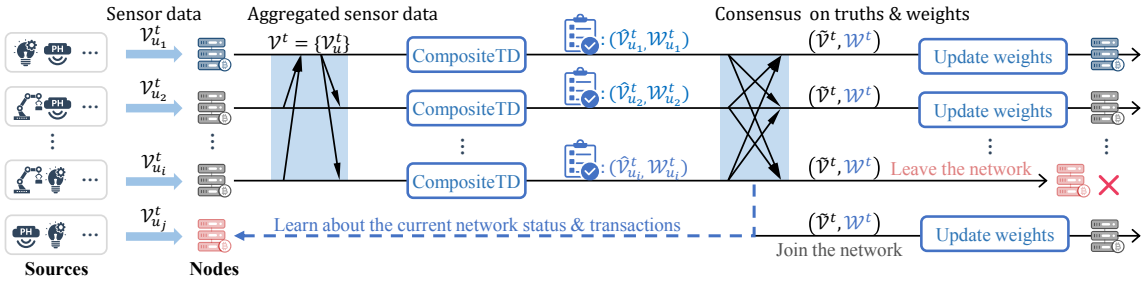



Fig. 1. The workflow of SenFEED. The leader node is highlighted in blue, normal nodes are grey, and nodes not in the network are red.

aggregation, where nodes send the received data to the leader node which aggregates and distributes aggregated sensor data $\mathcal{V}^t = \{\mathcal{V}^t_{u_i}\}_{u_i \in \mathcal{U}_c}$ to others. Meanwhile, the quality of each source may differ, where some even provide Byzantine data. Thus, we make nodes independently employ $\mathcal{V}^t$ and sources' priori weights $\mathcal{W}^{t-1}$ for CompositeTD to derive objects' truth $\hat{\mathcal{V}}^t_{u_i}$ and sources' weights $\mathcal{W}^t_{u_i}$. Then, each node $u_i$ broadcasts $(\hat{\mathcal{V}}^t_{u_i}, \mathcal{W}^t_{u_i})$, receives $\{(\hat{\mathcal{V}}^t_{u'}, \mathcal{W}^t_{u'})\}_{u' \in \mathcal{U}_c \setminus \{u_i\}}$ from others, and achieves consensus on objects' truths $\tilde{\mathcal{V}}^t$ and sources' weights $\mathcal{W}^t$ via the DPS-BFT protocol. Finally, they deliver the consensus result $(\tilde{\mathcal{V}}^t, \mathcal{W}^t)$.

Moreover, SenFEED also realizes a scalable and stable consensus protocol for nodes to join or leave the network. Any nodes can directly leave after the consensus, while the node (like $u_j$) waiting to join, must learn about the current network status and observe transactions in the off-chain consensus. It joins the network once the consensus is over and updates sources' weights in local CompositeTD with other nodes.

### C. Threat Model

The system's two major entities, sources and nodes, can be targets of manipulation by an adversary $\mathcal{A}$ through Byzantine influence. Consistent with existing studies [14], [16], [17], we assume that most nodes and sources are honest. $\mathcal{A}$ can manipulate no more than $\mathcal{F}_{\mathcal{U}_c}$ nodes where $\mathcal{F}_{\mathcal{U}_c} \leq \lfloor \frac{|\mathcal{U}_c|-1}{3} \rfloor$. For a given object $o$, $\mathcal{A}$ can manipulate no more than $\mathcal{F}_{\mathcal{S}_o}$ sources where $\mathcal{F}_{\mathcal{S}_o} \leq \lfloor \frac{|\mathcal{S}_o|}{2} \rfloor$. $\mathcal{A}$ can randomly select manipulated nodes and sources, but the number must be within the assumed range. Byzantine sources may spread Byzantine data about specific objects at a given timestamp and Byzantine nodes may tamper with objects' truths and sources' weights. Moreover, $\mathcal{A}$ can arbitrarily delay messages within the bounded latency. The nodes may join or leave the system, we assume the quorum consists of at least $\mathcal{Q}_{\mathcal{U}_c}$ nodes where $\mathcal{Q}_{\mathcal{U}_c} = \lceil \frac{|\mathcal{U}_c|+\mathcal{F}_{\mathcal{U}_c}+1}{2} \rceil$. It denotes the minimum number of nodes to achieve consensus on objects' truths and sources' weights.

The honest nodes and sources follow the workflow strictly. Honest sources are always available and accessible to nodes and the data from different honest sources are highly consistent. Honest nodes provide same objects' truth estimation and sources' weights, and do not provide conflicting information. The primary goal of SenFEED is to implement dynamic decentralized oracle services that provide accurate and real-time sensor data for smart contracts.

### D. Preliminaries

**Truth Discovery.** Truth discovery falls under the umbrella of multi-source data aggregation, aiming to identify the most accurate data from multiple, possibly conflicting sources by evaluating the reliability of the sources and the consistency of the data they provide. A node $u$ can obtain streaming data $\mathcal{V}^t$ broadcast by the leader node at timestamp $t$. Truth discovery allows node $u$ to derive objects' truth estimation and sources' weights: $\hat{\mathcal{V}}^t_u = \mathsf{t}(\mathcal{V}^t, \mathcal{W}^t_u)$ and $\mathcal{W}^t_u = \mathsf{f}(\sum_{o \in \mathcal{O}} \mathsf{d}(\mathcal{V}^t, \hat{\mathcal{V}}^t_u))$. The node $u$ evaluates the weight of each source $s$, *i.e.*, $w^t_{u,s} = \mathsf{f}(\sum_{o \in \mathcal{O}} \mathsf{d}(v^t_{s,o}, \hat{v}^t_{s,o}))$, and ultimately obtains $\mathcal{W}^t_u = \{w^t_{u,s}\}_{s \in \mathcal{S}}$. $\mathsf{t}(\cdot,\cdot)$ leverages sources' weights and sensor data at timestamp $t$ to derive objects' truths, $\mathsf{d}(\cdot,\cdot)$ measures the distance between the truth estimation of objects and the monitoring data about objects provided by sources, and function $\mathsf{f}(\cdot)$ is a monotonically decreasing function, implying that the larger the distance between $v^t_{s,o}$ and the truth estimation $\hat{v}^t_{s,o}$, the smaller weight is given to the source $s$.

**Partially Synchronous Byzantine Fault Tolerance.** Since sources and nodes are usually in relatively stable environments with controllable communication delays, we can adopt a widely studied consensus algorithm [28], [29] that has been used in decentralized systems with a partially synchronous network to tolerate faulty or malicious nodes. The extended version (with the same network setting), Dynamic BFT, was initially and formally proposed by Duan and Zhang [30]. *Dynamic* refers to the ability of the system to adapt to changes in the active nodes, allowing nodes to join or leave the network dynamically while maintaining consensus and fault tolerance. In a decentralized system consisting of $N_a$ active nodes, the dynamic BFT consensus guarantees consistency, liveness, and correctness [28], [30] even when up to $f_a$ $(N_a \geq 3f_a + 1)$ active nodes are compromised. The dynamic BFT is a critical component of SenFEED for nodes to achieve consensus.

## III. System Design

We now present the design details of SenFEED, which implements CompositeTD to enable a single node to perform accurate real-time truth inference on streaming data of IoT objects and develops the DPS-BFT protocol based on the dynamic decentralized oracle model to permit nodes to join or leave without compromising performance under the partially synchronous Byzantine-ridden network. Notably, we focus on extracting accurate and real-time sensor data from conflicting sources and achieving off-chain consensus among nodes, while

Authorized licensed use limited to: BEIJING UNIVERSITY OF POST AND TELECOM. Downloaded on January 26,2026 at 06:08:16 UTC from IEEE Xplore. Restrictions apply.

feeding data into smart contracts is an independent study (please refer to [18], [20]).

### A. Composite Truth Discovery Algorithm

We exploit the advantages of iterative-based and incremental-based approaches to develop the CompositeTD algorithm for accurate and real-time truth inference on IoT objects. Moreover, unlike existing truth discovery studies [27], [31]–[33], we consider the scenario where a strong adversary exists to manipulate a portion of nodes and sources at will.

**Iterative-based Truth Discovery.** The estimation of objects' truths and sources' weights alternates until convergence. The workflow of the iterative-based truth discovery for a single node is as follows. Each node $u$ receives sensor data (in streaming format) $\mathcal{V}_u^t$ in timestamp $t$ from accessible sources $\mathcal{S}_u$, and sends $\mathcal{V}_u^t$ to the leader node who aggregates sensor data and distributes aggregated sensor data $\mathcal{V}_t$ through the DPS-BFT protocol (in Section III-B). Agreement and liveness [30] features of the DPS-BFT protocol ensure that all nodes eventually obtain sensor data of IoT objects from all sources at timestamp $t$, *i.e.*, $\mathcal{V}^t = \{\mathcal{V}_u^t\}_{u\in\mathcal{U}_c}$. The goal of iterative-based truth discovery is formulated as follows:

$$\begin{aligned} &\text{Minimize} \sum_{t=1}^{T}\sum_{s\in\mathcal{S}}\sum_{o\in\mathcal{O}}(\mathsf{e}_u^t - \mathbb{1}_{s,o}\log w_{u,s}^t) \\ &\text{Subject to} \sum_{s\in\mathcal{S}}\log w_{u,s}^t = 1 \end{aligned} \tag{1}$$

where $\mathsf{e}_u^t = \gamma \cdot w_{u,s}^t \mathsf{d}(\mathbb{1}_{s,o}v_{s,o}^t, \mathbb{1}_{s,o}\hat{v}_o^t)$, $\mathsf{d}(\cdot,\cdot)$ can be instantiated as any distance function, $\gamma$ is the scale factor to represent the weight of the error, and $\log w_{u,s}^t$ prevents $w_{u,s}^t$ from equal 0. The principle of truth discovery is to assign high weights to sources that frequently provide reliable data. The object's truth (source's weight) is derived by weighted aggregation with fixed sources' weights (objects' truths):

$$\hat{v}_o^t = \frac{\sum_{s\in\mathcal{S}}\mathbb{1}_{s,o}w_{u,s}^t v_{s,o}^t}{\sum_{s\in\mathcal{S}}\mathbb{1}_{s,o}w_{u,s}^t} \quad \forall o \in \mathcal{O} \tag{2}$$

$$w_{u,s}^t = \mathsf{f}(\sum_{o\in\mathcal{O}}\mathsf{d}(\mathbb{1}_{s,o}v_{s,o}^t, \mathbb{1}_{s,o}\hat{v}_o^t)) \quad \forall s \in \mathcal{S} \tag{3}$$

Each node $u$ alternately executes Eq. (2) and Eq. (3) until convergence, and then broadcasts the estimation of objects' truths $\hat{\mathcal{V}}_u^t$ and sources' weights $\mathcal{W}_u^t$ across the network.

The iterative-based approach is accurate but fails to meet the real-time requirement. It requires alternating estimation of objects' truths and sources' weights at each timestamp until convergence, resulting in enormous computation and latency.

**Incremental-based Truth Discovery.** Objects' truths and sources' weights are updated continuously by incorporating arriving sensor data with the existing knowledge. Different from the iterative-based approach, the incremental-based truth discovery doesn't need iterative convergence, instead constructing the probabilistic model using prior knowledge of source weights and errors. The monitoring error of a source $s$ on an object $o$ at timestamp $t$ is denoted as $\Delta v_{s,o}^t = \sqrt{\mathsf{d}(\mathbb{1}_{s,o}v_{s,o}^t, \mathbb{1}_{s,o}\hat{v}_o^t)}$, which satisfies the normal distribution: $\Delta v_{s,o}^t | w_{u,s}^t \sim N(0, \frac{1}{\gamma w_{u,s}^t})$. Moreover, we define $\Delta\mathcal{V}_s^t = \{\Delta v_{s,o}^t\}_{o\in\mathcal{O}}$ to denote the monitoring error set of a source $s$ at timestamp $t$ and $\Delta\mathcal{V}_s^{1:T} = \bigcup_{t=1}^{T}\Delta\mathcal{V}_s^t$ to denote the monitoring error set of a source $s$ from timestamp 1 to timestamp $T$. Then, we transform the minimization problem into the maximum likelihood estimation of $\prod_{s\in\mathcal{S}} p(\Delta\mathcal{V}_s^{1:T}|w_{u,s}^T)$.

**Theorem 1.** *Given objects' truths $\hat{\mathcal{V}}_u^{1:T} = \bigcup_{t=1}^{T}\hat{\mathcal{V}}_u^t$ from timestamp 1 to $T$ and the probability distribution of the error $p(\Delta v_{s,o}^t|w_{u,s}^T)$, the minimization problem in Eq. (1) is equivalent to the maximum likelihood estimation of $\prod_{s\in\mathcal{S}} p(\Delta\mathcal{V}_s^{1:T}|w_{u,s}^T)$.*

*Proof.* First, we formulate the mathematical expression.

$$\begin{aligned} \prod_{s\in\mathcal{S}} p(\Delta\mathcal{V}_s^{1:T}|w_{u,s}^T) &= \prod_{t=1}^{T}\prod_{s\in\mathcal{S}}\prod_{o\in\mathcal{O}} p(\Delta v_{s,o}^t|w_{u,s}^T) \\ \propto \prod_{t=1}^{T}\prod_{s\in\mathcal{S}}(\sqrt{\gamma w_{u,s}^T})^{\sum_{o\in\mathcal{O}}\mathbb{1}_{s,o}} &e^{\frac{-\gamma w_{u,s}^T\sum_{o\in\mathcal{O}}(\Delta v_{s,o}^t)^2}{2}} \end{aligned} \tag{4}$$

We transform it to obtain the maximum likelihood estimation.

$$\begin{aligned} &-\log(\prod_{s\in\mathcal{S}} p(\Delta\mathcal{V}_s^{1:T}|w_{u,s}^T)) \\ \propto\ &\frac{1}{2}\sum_{t=1}^{T}\sum_{s\in\mathcal{S}}\sum_{o\in\mathcal{O}}(\gamma\cdot w_{u,s}^T(\Delta v_{s,o}^t)^2 - \mathbb{1}_{s,o}(\log w_{u,s}^T - \log\gamma)) \\ \propto\ &\sum_{t=1}^{T}\sum_{s\in\mathcal{S}}\sum_{o\in\mathcal{O}}(\gamma\cdot w_{u,s}^T\mathsf{d}(\mathbb{1}_{s,o}v_{s,o}^t, \mathbb{1}_{s,o}\hat{v}_o^t) - \mathbb{1}_{s,o}\log w_{u,s}^T) \end{aligned} \tag{5}$$

Thus, we prove that minimizing Eq. (1) equals maximizing Eq. (4), *i.e.*, Theorem 1 holds. □

Consistency with existing research [32], [33], we set $w_{u,s}^t$ to satisfy the Gamma distribution: $p(w_{u,s}^t) \propto \Gamma(\alpha,\beta)$ where $\alpha$ is the shape parameter and $\beta$ is the scale parameter. Combined with Theorem 1, the prior distribution of the source $s$ at the latest timestamp $T$ satisfies $p(w_{u,s}^T|\Delta\mathcal{V}_s^{1:T}) \sim \Gamma(\alpha + \frac{\sum_{t=1}^{T}\sum_{o\in\mathcal{O}}\mathbb{1}_{s,o}}{2}, \beta + \frac{\gamma\cdot\prod_{t=1}^{T}\sum_{o\in\mathcal{O}}(\Delta v_{s,o}^t)^2}{2})$. Thus, we obtain the maximum a posteriori (MAP) estimation:

$$w_{u,s}^T = \frac{-2 + 2\alpha + \sum_{t=1}^{T}\sum_{o\in\mathcal{O}}\mathbb{1}_{s,o}}{2\beta + \gamma\cdot\prod_{t=1}^{T}\sum_{o\in\mathcal{O}}(\Delta v_{s,o}^t)^2} \tag{6}$$

Combined with Eq. (2), the MAP estimation of sources' weights at timestamp $t$ is used to derive objects' truth at timestamp $t+1$ with a single computation where the estimation does not require iterative computation.

The incremental-based approach is efficient but inaccurate because it struggles to adapt objects' truths or data distribution changes and errors from earlier stages propagate and compound, leading to a gradual decline in accuracy over time. Thus, it may fail to meet the accuracy requirement in IoT.

**Composite Truth Discovery.** In realistic scenarios, sources' weights tend to be relatively stable for long periods where incremental-based truth discovery can obtain accurate objects' truths in real-time. However, when sources' weights fluctuate greatly, iterative-based truth discovery proves more effective in maintaining accuracy. Therefore, we further propose CompositeTD to meet both accuracy and real-time requirements.

Authorized licensed use limited to: BEIJING UNIVERSITY OF POST AND TELECOM. Downloaded on January 26,2026 at 06:08:16 UTC from IEEE Xplore. Restrictions apply.

To implement the CompositeTD, it's crucial to accurately evaluate the fluctuations in sources' weights. We define source weight evolution $\Delta w_{u,s}^t = |w_{u,s}^t - w_{u,s}^{t-1}|$, which represents the absolute difference in sources' weights at neighboring timestamps. We leverage $\Delta w_{u,s}^t$ to determine whether sources' weights suddenly fluctuate greatly. When timestamp $t = 1$ or the source weight evolution exceeds the threshold ($\Delta w_{u,s}^t > \frac{\sqrt{\varepsilon}}{|\mathcal{S}|}$), the CompositeTD initially alternately executes Eq. (2) and Eq. (3) until convergence to obtain the accurate sources' weights. In the next timestamp, we leverage the latest sources' weights to derive objects' truths based on Eq. (2) and obtain the MAP estimation of sources' weights based on Eq. (6). Since sensor data for most IoT objects usually evolves smoothly, we take objects' history truths as a virtual data source with the fixed weight $\widetilde{w}$ and incorporate historical data into Eq. (2) to capture the smoothing characteristic:

$$\hat{v}_o^t = \frac{\sum_{s\in\mathcal{S}} \mathbb{1}_{s,o} w_{u,s}^t v_{s,o}^t + \tilde{w}\hat{v}_o^{t-1}}{\sum_{s\in\mathcal{S}} \mathbb{1}_{s,o} w_{u,s}^t + \tilde{w}} \tag{7}$$

In the following, we conduct the theoretical analysis of unit error and cumulative error in the CompositeTD. The unit error of the object truth is $\phi_o^t = (\frac{\sum_{s\in\mathcal{S}_o} v_{s,o}^t (w_{u,s}^t - w_{u,s}^{t-1})}{\max\{v_{s,o}^t\}_{s\in\mathcal{S},o\in\mathcal{O}}})^2$ and the threshold of the unit error is $\varepsilon$.

**Theorem 2.** *If* $\Delta w_{u,s}^t \le \frac{\sqrt{\varepsilon}}{|\mathcal{S}|}(\forall s \in \mathcal{S})$, *then* $\phi_o^t \le \varepsilon(\forall o \in \mathcal{O})$.

*Proof.* With the mathematical equations of $\Delta w_{u,s}^t$ and $\phi_o^t$, we derive the unit error for object $o$:

$$\sqrt{\phi_o^t} \le \sum_{s\in\mathcal{S}_o} |\frac{v_{s,o}^t (w_{u,s}^t - w_{u,s}^{t-1})}{\max\{v_{s,o}^t\}_{s\in\mathcal{S}_o,o\in\mathcal{O}}}| \le \sum_{s\in\mathcal{S}_o} \Delta w_{u,s}^t \le \sqrt{\varepsilon} \tag{8}$$

Therefore, we prove that $\phi_o^t \le \varepsilon(\forall o \in \mathcal{O})$ when $\Delta w_{u,s}^t \le \frac{\sqrt{\varepsilon}}{|\mathcal{S}|}(\forall s \in \mathcal{S})$. □

**Theorem 3.** *If* $\Delta w_{u,s}^t \le \frac{\sqrt{\varepsilon}}{|\mathcal{S}|}(\forall s \in \mathcal{S}, t \in (i,j])$, *then* $\Phi_o^{i,j} = \sum_{t=i+1}^{j} \phi_o^t \le \Delta T \cdot \varepsilon(\forall o \in \mathcal{O}, \Delta T = j - i)$.

*Proof.* Let $\Delta T = j - i$, with the Theorem 2, we derive that

$$\begin{aligned} \Phi_o^{i,j} &= \sum_{t=i+1}^{j} (\frac{\sum_{s\in\mathcal{S}_o} v_{s,o}^t (w_{u,s}^t - w_{u,s}^{t-1})}{\max\{v_{s,o}^t\}_{s\in\mathcal{S},o\in\mathcal{O}}})^2 \\ &\le \sum_{t=i+1}^{j} (\sum_{s\in\mathcal{S}_o} \Delta w_{u,s}^t)^2 = \Delta T \cdot \varepsilon \end{aligned} \tag{9}$$

Thus, we prove that $\Phi_o^{i,j} \le \Delta T \cdot \varepsilon(\forall o \in \mathcal{O})$ when $\Delta w_{u,s}^t \le \frac{\sqrt{\varepsilon}}{|\mathcal{S}|}(\forall s \in \mathcal{S}, t \in (i,j])$. □

Theorem 2 and Theorem 3 prove that the CompositeTD can leverage sources' weights (derived by alternately executing Eq. (2) and Eq. (3) until convergence) as the prior distributions to obtain accurate objects' truths in successive timestamps without iterative computation. Moreover, it can detect fluctuations in source weights when Byzantine attacks occur by the source weight evolution and then alternates objects' truths estimation and sources' weights estimation until convergence to obtain accurate results. Thus, the CompositeTD meets both accuracy and real-time requirements in IoT scenarios.

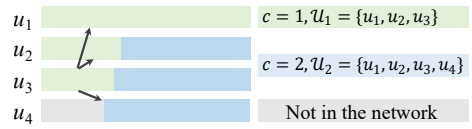



Fig. 2. Liveness problems. $u_3$ is the leader node from $c = 1$ to $c = 2$.

### B. Dynamic Partially Synchronous BFT Protocol

For a BFT system, when nodes join or leave, *system reconfiguration* [34] is usually required, which may result in zero throughput; otherwise, *liveness problems* may arise: 1) ***The network status maintained by nodes may be inconsistent.*** As shown in Fig. 2, node $u_4$ requests to join the network when $c = 1$, then $u_2$ and $u_3$ update the network status from $c = 1$ to $c = 2$, but $u_1$ doesn't catch up with the network status and remains at $c = 1$. 2) ***View-change may halt forever.*** The view-change occurs when $c = 2$ and node $u_4$ will be the new leader node. However, node $u_1$ (still at $c = 1$) doesn't realize that $u_4$ has joined the network. In this case, $u_1$ considers himself the new leader while $u_4$ expects to receive $\mathcal{Q}_{\mathcal{U}_2} = 3$ messages to start a new view as the leader. Thus, the network halts forever in the view-change and can't process any messages.

To solve these challenges, we develop the DPS-BFT protocol for the dynamic decentralized oracle model, which forces nodes to perform the network status discovery mechanism to catch up with the network status and leverage the reliable view-change mechanism to change the leader node. With the protocol, we implement scalable, stable, and reliable oracle services, even if malicious nodes and sources are mixed in.

We make some necessary assumptions and constraints on our model: 1) Nodes joining the network $\mathcal{U}_{\text{join}}$ and nodes leaving the network $\mathcal{U}_{\text{leave}}$ are bounded when the network status is updated from $\mathcal{U}_c$ to $\mathcal{U}_{c+1}$. 2) The maximum number of Byzantine nodes (faulty or malicious) is $\mathcal{F}_{\mathcal{U}_c}$ when the network status is $\mathcal{U}_c$. 3) Consistent with Chainlink [14], the most popular commercial product for oracle services, our model is a partially synchronous network where messages will be delivered within some unknown but bounded time.

**The network status discovery mechanism.** Each round of consensus consists of a batch of requests (including truth inference requests and node requests), and node requests (join/leave requests) are processed after truth inference requests. The mechanism requires each node to maintain an ordered *request list* which contains delivered requests. A request is considered delivered only if more than two-thirds of nodes agree on it, *i.e.*, each delivered request is signed by $\mathcal{Q}_{\mathcal{U}_c}$ nodes (the network status is $\mathcal{U}_c$). We introduce *node request list* (abbreviated as OList), which contains only node requests ordered by the network status number $c$, and any node $u_i$ can prove his network status to others by OList. Before updating the network status, active nodes in the network leverage the *temporary set* to store the nodes that request to join. Thus, the number of Byzantine nodes is still $\mathcal{F}_{\mathcal{U}_c}$ and the quorum is still $\mathcal{Q}_{\mathcal{U}_c} = \lceil \frac{|\mathcal{U}_c| + \mathcal{F}_{\mathcal{U}_c} + 1}{2} \rceil$ until the network status is updated from $c$ to $c+1$. The nodes in the temporary set will formally join the network when the network status is updated to $c+1$. As shown in Algorithm 1, node $u_i$ (including nodes in the temporary set)

Authorized licensed use limited to: BEIJING UNIVERSITY OF POST AND TELECOM. Downloaded on January 26,2026 at 06:08:16 UTC from IEEE Xplore. Restrictions apply.

**Algorithm 1** Network status discovery algorithm.

**Function func_obtain_netstat():**
  broadcast $\langle \text{request_netstat}, c \rangle$;
  start a timer $t$;
  **while** $t < \Delta$ **do**
    receive $\langle \text{netstat}, c', \mathcal{U}_{c'}, \mathsf{OList}' \rangle$ from **func_send_netstat()**;
    **if** $\mathsf{OList}'$ is valid & $c' > c$ **then**
      $c = c', \mathcal{U}_c = \mathcal{U}_{c'}, \mathsf{OList} = \mathsf{OList}'$;
  **return** $c, \mathcal{U}_c, \mathsf{OList}$
**Function func_send_netstat():**
  receive $\langle \text{request_netstat}, c' \rangle$;
  reply $\langle \text{netstat}, c, \mathcal{U}_c, \mathsf{OList} \rangle$;
**Function func_request_timeout():**
  $c', \mathcal{U}_{c'}, \mathsf{OList}'$=func_obtain_netstat();
  **if** $c = c'$ **then**
    func_view_change();
  **else**
    broadcast $\langle \text{update}, c, s, i \rangle$;
    receive $\mathcal{Q}_{\mathcal{U}_{c'}}$ $\langle \text{update_reply}, \mathsf{RList}, j \rangle$;
    deliver requests in $\mathsf{RList}$;
  **if** $j \in \mathcal{U}_{c'}$ & $c > c'$ **then**
    send $\langle \text{update_reply}, \mathsf{RList}, j \rangle$ to $u_j$;

broadcasts $\langle \text{request_netstat}, c \rangle$ to obtain the current network status. Then, $u_i$ receive reply $\langle \text{netstat}, c', \mathcal{U}_{c'}, \mathsf{OList}' \rangle$ from other nodes where $c'$ is the number of network status. $u_i$ determines whether network status $\mathcal{U}_{c'}$ is valid and treats the newest valid network status as the current network status. The mechanism terminates when the execution time exceeds $\Delta$.

Although the mechanism enables nodes to learn about the current network status, they do not provide a method for nodes to be aware that their network status has expired. To keep all nodes catching up with the current network status, nodes set a time limit for each request. If node $u_i$ times out when processing a request, his network status may have expired. For every request, $u_i$ needs to collect $\mathcal{Q}_{\mathcal{U}_c}$ matching messages to deliver the request, but some active nodes (in the view of $u_i$) may have left the network. It is necessary to perform the func_obtain_netstat() to obtain the network status. If $c' = c$, it means that the leader node is manipulated or offline, causing the request timeout. It is necessary to conduct view-change to select a new leader node. Otherwise, the network status of $u_i$ has expired and the current network status is $\mathcal{U}_{c'}$. Then, $u_i$ needs to broadcast $\langle \text{update}, c, s, i \rangle$ where $s$ is the stable checkpoint [28] and wait for the reply $\langle \text{update_reply}, \mathsf{RList}, j \rangle$ from other nodes. Until $\mathcal{Q}_{\mathcal{U}_{c'}}$ messages are received, $u_i$ can confirm results from the timeout request to the latest request, thus catching up with other nodes. When the node $u_i$ receives $\langle \text{update}, c', s, j \rangle$, he checks the $u_j$ whether in $\mathcal{U}_{c'}$. If his network status is newer than $u_j$, then he sends $\langle \text{update_reply}, \mathsf{RList}, j \rangle$ to $u_j$.

**The reliable view-change mechanism.** The leader node is required to be honest and catch up with the current network status. The reliable view-change mechanism is leveraged to change the leader when he is manipulated or time out. As shown in Algorithm 2, a node $u_i$ modifies the view number $v$ to $v + 1$. Then, he broadcasts the view-change message $\langle \text{view_change}, v, c, s, \mathsf{RList}, \mathcal{U}_c, i \rangle$. When he receive $\langle \text{view_change}, v, c', s, \mathsf{RList}, \mathcal{U}_{c'}, j \rangle$ from node $u_j$, he will

**Algorithm 2** Reliable view-change algorithm.

**Function func_view_change():**
  v=v+1;
  broadcast $\langle \text{view_change}, v, c, s, \mathsf{RList}, \mathcal{U}_c, i \rangle$;
  receive $\langle \text{view_change}, v, c', s, \mathsf{RList}, \mathcal{U}_{c'}, j \rangle$;
  **if** $c' < c$ **then**
    send $\langle \text{view_change}, v, c', s, \mathsf{RList}, \mathcal{U}_{c'}, j \rangle$ to $\mathcal{U}_c / \mathcal{U}_{c'}$;
  **else**
    $\mathcal{U}_c = \mathcal{U}_{c'}$;
    send $\langle \text{view_change}, v, c, s, \mathsf{RList}, \mathcal{U}_c, i \rangle$ to $\mathcal{U}_c$;
**Function func_start_new_view():**
  receive $\mathcal{U}_c$ $\langle \text{view_change}, v, c', s, \mathsf{RList}, \mathcal{U}_{c'}, j \rangle$;
  broadcast $\langle \text{new_view}, v, c, \mathsf{VC} \rangle$;

compare $c$ and $c'$. If $c' < c$, it means that the network status of $u_i$ is newer and he delivers the view-change message from $u_j$ to nodes in $\mathcal{U}_c$ that have not receive the message. Otherwise, the network status of $u_j$ is newer, and $u_i$ updates the network status and rebroadcasts the view-change message to active nodes. Through the network status discovery mechanism, $u_i$ can determine that $c$ is the latest. Thus, the new leader node is $u_{[v\%|\mathcal{U}_c|]}$. When the new leader receives $\mathcal{Q}_{\mathcal{U}_c}$ view-change messages, he broadcasts $\langle \text{new_view}, v, c, \mathsf{VC} \rangle$ to start the new view, where $\mathsf{VC}$ denotes view-change messages from other nodes and denotes pending requests.

### C. Theoretical Analysis

We now prove that SenFEED can defend the strong adversary in Section II-C and solve liveness problems.

**Theorem 4.** *SenFEED is resilient to dynamic manipulation of sources and nodes by the adversary $A$.*

*Proof.* If the data source $s \in \mathcal{S}$ suddenly becomes Byzantine, the data from $s$ can deviate arbitrarily from objects' truths. In this case, the monitoring error $\Delta\mathcal{V}_s^t$ generated by the byzantine source $s$ may increase significantly. The CompositeTD algorithm derives objects' truths based on Eq. (2) and obtains the MAP estimation of sources' weights based on Eq. (6) according to the monitoring error. Then, a lower weight is assigned to $s$, which in turn leads to the increase of $\Delta w_{u,s}^t$. Once $\Delta w_{u,s}^t > \frac{\sqrt{\varepsilon}}{|\mathcal{S}|}$, nodes alternately execute Eq. (2) and Eq. (3) until convergence to obtain more accurate objects' truths and sources' weights. Thus, the Byzantine source $s$ that is suddenly manipulated by $\mathcal{A}$ is detected by nodes and assigned a weight close to 0 that has little effect on objects' truths. It is important to note that the detection capability is based on more than half of the data sources and more than two-thirds of nodes being honest, which makes SenFEED resilient to dynamic manipulation of data sources by the adversary $A$.

The agreement and liveness features of the BFT protocol ensure that all nodes will eventually obtain the streaming data of IoT objects from all sources at timestamp $t$. The signature is utilized to ensure that the data is non-tampered. For each round of truth discovery, each node performs CompositeTD independently and broadcasts the results (including objects' truths $\hat{\mathcal{V}}_u^t$ and sources' weights $\hat{\mathcal{W}}_u^t$). Nodes achieve consensus on objects' truths and sources' weights when they receive $\mathcal{F}_{\mathcal{U}_c} + 1$ signed messages with the same timestamp and result.

Authorized licensed use limited to: BEIJING UNIVERSITY OF POST AND TELECOM. Downloaded on January 26,2026 at 06:08:16 UTC from IEEE Xplore. Restrictions apply.

Nodes that deviate from the consensus result are marked as Byzantine nodes by others. Moreover, each node sets a timer for each request, if the request is timeout, his network status may be expired or the leader node may be manipulated by $\mathcal{A}$ to delay the message. It will perform func_obtain_netstat() to catch up with the network status. If the network status is up-to-date, then the leader node is malicious. Most honest nodes achieve consensus to conduct view-change, thus terminating the malicious behavior of the leader node. If the node turns Byzantine (times out or deviates from consensus) for a long time, it will be removed from the network. Thus, SenFEED is also resilient to dynamic manipulation of nodes by $A$. □

**Theorem 5.** *SenFEED solves liveness problems when nodes join or leave the network without the system reset.*

*Proof.* Assume that the network status numbered $c$ is up-to-date. The initial nodes $\mathcal{U}_0$ in the network are known for all oracles. Nodes use OList to verify the network status, that contains node requests ordered by the network status number (denoted as $\{\mathsf{OList}_i\}_{0\leq i\leq c}$). We assume that node $u_1$ has $\mathsf{OList}^1$ and node $u_2$ has $\mathsf{OList}^2$ where $\mathsf{OList}^1$ is equal to $\mathsf{OList}^2$ when $0 \leq i < k$ $(0 < k \leq c)$. $\mathcal{U}_k$ is the first network status where $\mathsf{OList}^1$ is not equal to $\mathsf{OList}^2$. $\mathsf{OList}_k^1$ and $\mathsf{OList}_k^2$ are signed and delivered by more than $\mathcal{Q}_{\mathcal{U}_k}$ nodes, respectively. Thus, at least one correct node signs both $\mathsf{OList}_k^1$ and $\mathsf{OList}_k^2$. However, correct nodes cannot exhibit such malicious behavior. Thus, the legitimate OList is unique. For each node request in $\mathsf{OList}_i$, it is signed and delivered by more than $\mathcal{Q}_{\mathcal{U}_i}$ nodes [28]. Nodes can obtain $\mathcal{U}_1$ based on $\mathcal{U}_0$ and $\mathsf{OList}_0$ as $\mathsf{OList}_0$ consists some leave/join requests during the network status $\mathcal{U}_0$. Similarly, nodes can obtain $\mathcal{U}_c$ based on $\mathcal{U}_{c-1}$ and $\mathsf{OList}_{c-1}$. To catch up with the network status, node $u$ can broadcast $\langle$request_netstat$\rangle$ to others by func_obtain_netstat(). Upon nodes receive the request, they reply $\langle \text{netstat}, c, \mathcal{U}_c, \mathsf{OList}\rangle$. Node $u$ leverages OList to verify node requests and eventually catches up with the network status by calling the function multiple times. When new nodes join, they can catch up with the network status by obtaining no less than $\mathcal{Q}_{\mathcal{U}_c}$ $\langle$netstat$\rangle$ from others. Moreover, when the leader node is malicious, honest nodes can obtain the current network status in the same way and broadcast $\langle$view_change$\rangle$ request to select a new honest leader. Then, they will receive $\mathcal{Q}_{\mathcal{U}_{c'}}$ $\langle$view_change$\rangle$ from others and achieve consensus on the new leader node. The new leader in $\mathcal{U}_c$ starts the new view by broadcasting $\langle$new_view$\rangle$. With these mechanisms, active nodes in SenFEED can update $\mathcal{U}_c$ promptly when other nodes join or leave to avoid the inconsistent network status and view-change halt, thus solving the liveness problems without the system reset. □

## IV. Experimental Evaluation

### A. Experimental Setup

To implement SenFEED prototype, we incorporate the CompositeTD algorithm and the DPS-BFT protocol into the PBFT prototype [35] implemented in Python. SenFEED prototype is deploy on a Linux Ubuntu 20.04.3 LTS system with an Intel Core I9-13900HX (2.2 GHz) and 32GB of RAM. We create a simulation environment that employs partially synchronous configuration and assigns ports of the system to sources (IoT sensors) and nodes. Sources send streaming data to nodes via HTTP with 100 Mb/s bandwidth and nodes synchronize streaming data via broadcast. We assume an unknown maximum message delay time $m \leq 0.8$s in the simulation environment. Sources $\mathcal{S} \in [10, 250]$ monitor different objects $|\mathcal{O}| \in [4, 30]$ in real-time, package their identifier number, message number and streaming data into a message, and communicate with nodes through their unique ports. There are $|\mathcal{U}_c| \in [4, 240]$ nodes running independently in simulation environment. The number of Byzantine nodes $\mathcal{F}_{\mathcal{U}_c}$ is no more than $\lfloor \frac{|\mathcal{U}_c|-1}{3} \rfloor$. We compare SenFEED with Chainlink [14] (the most popular commercial product) and DecenTruth [16] (state-of-the-art research on decentralized oracle services). Chainlink aggregates data from multiple nodes on-chain and feeds the median as the final data; DecenTruth enables nodes to independently perform iterative-based truth discovery off-chain and agree on objects' truths by consensus protocol. We utilize Ethereum [13] smart contracts to aggregate objects' truths on-chain for Chainlink and leverage HoneyBadgerBFT [36] to achieve consensus on objects' truths off-chain for DecenTruth.

**Dataset.** We conduct experiments in a near-realistic IoT scenario with two real-world datasets and one synthetic dataset. Weather Dataset [37] includes temperature data for one week from 18 weather forecast sources for 30 cities in America, where Accuweather.com is the ground truth [37]. Sensor DataSet [38] includes monitoring data (updated every 30 seconds) from 54 sensors in the Intel Berkeley Research lab, including temperature, humidity, light, and voltage. We use the normal distribution to expand Sensor DataSet to obtain more sensor data and compare the efficiency of different truth discovery algorithms in IoT scenarios. Since Sensor DataSet doesn't have the ground truth, we generated a Synthetic Dataset that includes monitoring data from 250 sensors on 20 IoT objects (five times the scale of Sensor DataSet [38] and updated every 10 seconds). In the Synthetic Dataset, sensors generate large amounts of streaming data that are transmitted in real-time, while Byzantine sources and nodes randomly deviate from the expected behavior. The monitoring data $v_{s,o}^t$ of sensor $s$ at timestamp $t$ for object $o$ satisfies $N(\hat{v}_o^t, \sigma_o)$, where $\hat{v}_o^t$ is the ground truth of the object $o$ and $\sigma_o$ represents the unpredictability of sources, *i.e.*, Byzantine sources can arbitrarily deviate from ground truths.

**Baseline.** TruthFinder [39] and CBI-TD [16] are iterative-based truth discovery designed to identify the most reliable information from conflicting sources by evaluating the trustworthiness of each source. DynaTD [32] is incremental-based truth discovery that updates and identifies accurate information from dynamic sources incrementally using advanced statistical methods without iterative computation. Considering the smoothing factor and decay factor, some extensions of DynaTD are proposed, *i.e.*, DynaTD+smoothing, DynaTD+Decay, and DynaTD+All. Mean and Median are basic approaches that ignore differences in source reliability.

Authorized licensed use limited to: BEIJING UNIVERSITY OF POST AND TELECOM. Downloaded on January 26,2026 at 06:08:16 UTC from IEEE Xplore. Restrictions apply.

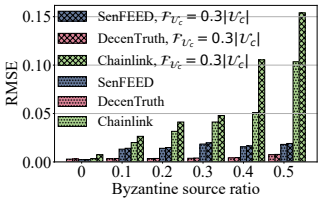



Fig. 3. RMSE evolves with different Byzantine source ratios.

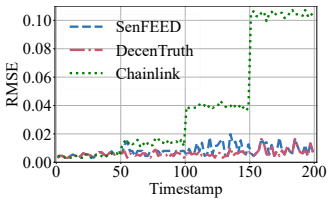



Fig. 4. RMSE evolves with time.

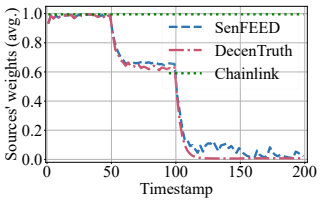



Fig. 5. Byzantine sources' average weights evolve with time.

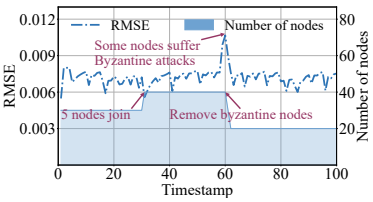
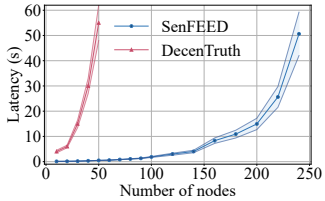
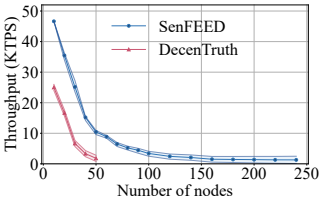



Fig. 6. Detect Byzantine nodes.

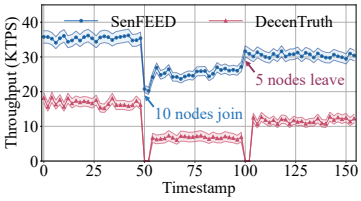



(a) Latency Comparison

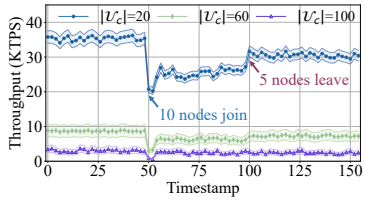



(b) Throughput Comparison

Fig. 7. Performance Comparison of SenFEED and DecenTruth [16].




(a) SenFEED vs DecenTruth [16]




(b) Different number of service nodes

Fig. 8. Impact on throughput when some nodes leave or join the system.

We adopt the Root Mean Squared Error (RMSE) to measure the differences between estimate truths and ground truths, providing a clear indication of the algorithms' accuracy.

### *B. Evaluation*

**Comparison of accuracy and efficiency.** As shown in Table I, the efficiency of CompositeTD is higher than CBI-TD by approximately $140.76\%$ and the accuracy is close to CBI-TD in Weather Dataset. In Sensor Dataset, CompositeTD is $5\times$ and $12\times$ more efficient than CBI-TD and TruthFinder. In Synthetic Dataset, although CompositeTD takes $4$ seconds longer for truth discovery per round compared to non-iterative approaches, it still meets the real-time requirement as objects' truths update every $10$ seconds. Notably, the accuracy of CompositeTD is comparable to CBI-TD, and it achieves the RMSE of only $2.4\%$ of DynaTD and two orders of magnitude smaller than Mean and Median. Furthermore, the efficiency of CompositeTD surpasses CBI-TD by an order of magnitude. Although Mean and Median are the most efficient methods, they suffer from poor accuracy due to equal treatment of sources. Incremental-based approaches, though considering source reliability, fail to adapt to sudden changes, leading to suboptimal accuracy. Iterative-based approaches like CBI-TD are accurate but inefficient, and TruthFinder struggles with streaming data accuracy. CompositeTD overcomes these limitations and achieves both accuracy and real-time requirements, more suitable for IoT applications.

**Byzantine resilience evaluation.** In the Synthetic Dataset, we compare the long-term accuracy of truth estimation of SenFEED (using CompositeTD), Chainlink (using Median), and DecenTruth (using CBI-TD) under different Byzantine sources ratio and Byzantine nodes ratio as shown in Fig. 3. The experimental results on the Synthetic Dataset demonstrate that SenFEED and DecenTruth maintain high long-term accuracy with low RMSE, even as the Byzantine source ratio increases. In contrast, Chainlink shows a significant rise in RMSE with higher Byzantine source ratios, indicating its vulnerability to Byzantine attacks. SenFEED, benefiting from its consensus protocol, effectively mitigates the impact of Byzantine nodes based on the majority of honest nodes, thus achieving superior accuracy and robustness compared to Chainlink.

TABLE I
COMPARISON OF ACCURACY & EFFICIENCY.

| Algorithm | Weather | | Sensor | Synthetic | |
|---|---|---|---|---|---|
| | RMSE | Time (s) | Time (s) | RMSE | Time (s) |
| CompositeTD | 0.9110 | 0.0157 | 3.9549 | 0.0835 | 4.4888 |
| CBI-TD | **0.9030** | 0.0378 | 19.6733 | **0.0251** | 45.9098 |
| TruthFinder | 1.2282 | 0.7340 | 43.8299 | 10.1369 | 86.9221 |
| DynaTD | 0.9622 | 0.0089 | 1.0201 | 3.4807 | 0.6418 |
| DynaTD+Decay | 0.9592 | 0.0093 | 1.2285 | 3.3645 | 0.6481 |
| DynaTD+Smoothing | 0.9550 | 0.0091 | 1.2493 | 3.3573 | 0.7084 |
| DynaTD+All | 0.9521 | 0.0091 | 1.1069 | 3.3460 | 0.6825 |
| Mean | 1.2900 | 0.0085 | 0.9191 | 10.1369 | **0.6157** |
| Median | 0.9824 | **0.0056** | **0.9088** | 13.6676 | 0.6279 |

We further show the process of SenFEED and DecenTruth to defend against Byzantine attacks as shown in Fig. 4 and Fig. 5. Fig. 4 illustrates the RMSE evolution for SenFEED, DecenTruth, and Chainlink when the timestamp $t$ goes from $0$ to $200$. When $t = 50$ ($t = 100$), $20\%$ ($40\%$) of sources suffer Byzantine attacks, both suffer short-term accuracy decreases and recover quickly, while Chainlink's accuracy becomes increasingly worse. As shown in Fig. 5, SenFEED and DecenTruth effectively defend against Byzantine attacks by rapidly detecting and assigning lower weights to Byzantine sources. When $t = 150$, $30\%$ nodes suffer Byzantine attacks, SenFEED and DecenTruth are almost unaffected, while Chainlink's RMSE rises significantly. Thus, SenFEED and DecenTruth can detect Byzantine sources and nodes to defend against Byzantine attacks, while Chainlink has difficulty defending against such attacks.

Moreover, we demonstrate the process of SenFEED to detect Byzantine nodes in Fig. 6. When $t = 30$, $5$ nodes join the system for obtaining objects' truths in real-time. When $t = 58$, some nodes suffer Byzantine attacks, leading to a significant increase in RMSE. Then, SenFEED detects Byzantine nodes and removes them (the number of Byzantine nodes is $10$) from the system in $t = 60$, then RMSE drops to its original value (*i.e.*, accuracy recovery). Thus, SenFEED enables nodes to

Authorized licensed use limited to: BEIJING UNIVERSITY OF POST AND TELECOM. Downloaded on January 26,2026 at 06:08:16 UTC from IEEE Xplore. Restrictions apply.

join the system and effectively removes Byzantine nodes to maintain high accuracy, showcasing superior scalability and stability compared to DecenTruth and Chainlink.

**Off-chain consensus evaluation.** Since Chainlink employs on-chain consensus and is weakly defended against Byzantine attacks, we do not consider it for the off-chain consensus evaluation. Thus, we only employ SenFEED and DecenTruth for IoT scenarios and compare their latency and throughput. Fig. 7(a) compares the latency between SenFEED and DecenTruth for different number of nodes. As the number of nodes increases, the latency of DecenTruth increases dramatically, only supporting scales of a few dozen nodes. In contrast, the latency increase of SenFEED is not significant when nodes are less than $100$, significantly reducing the latency of node services. Since DecenTruth concentrates on Byzantine resilience and consensus latency without throughput, we try to implement DecenTruth to measure the throughput and the comparison result is shown in Fig. 7(b). As the number of nodes increases, the throughput of DecenTruth decreases dramatically as the consensus latency grows quadratically, significantly limiting requests that can be processed in one second and struggling to support systems with more than $50$ nodes. The throughput of SenFEED starts at around $48k$ transactions per second (TPS) and gradually decreases to approximately $10k$ TPS as node counts increase to $50$. As the number of nodes increases, even beyond $100$ nodes, the throughput of SenFEED stabilizes around $3k$ TPS. Thus, SenFEED effectively supports large-scale nodes and meets both real-time and throughput requirements of IoT scenarios.

**Dynamic evaluation.** Fig. 8(a) further shows the impact on throughput when some nodes leave or join the system when $\mathcal{U}_c = 20$. When $t = 50$ ($t = 100$), the throughput of DecenTruth drops to $0$ due to the need for a system reset when $10$ nodes join ($5$ nodes leave) the system. For SenFEED, since nodes need to synchronize with each other (especially the new nodes), the throughput drops to $56.3\%$ of that original, but can recover after several timestamps. When some nodes leave the system, the throughput of SenFEED does not change significantly as these nodes can directly leave the system after they deliver the leave request without synchronization operations. In Fig. 8(b), we further set up different initial numbers of nodes $|\mathcal{U}_c|$ for SenFEED. The experiment shows that SenFEED maintains near-normal throughput across different initial node counts when some nodes leave or join the system. Thus, SenFEED permits nodes to join or leave the network without compromising performance.

## V. Related Work

**IoT-blockchain.** The integration of blockchain technology into IoT has led to significant advancements, particularly through the application of smart contracts [10], [40], [41]. In [10], smart contracts are triggered to execute parametric insurance based on rainfall data provided by sensors for transparent claim settlements, thus reducing potential disputes. Viriyasitavat *et al.* [40] employ smart contracts to realize the service level agreement enforcement and data exchange for Quality of Service in IoT. Moreover, smart contracts are leveraged to realize fair exchanges for electricity/service between vehicles and the grid with privacy guarantees [41]. However, the issue of how smart contracts proactively access real-time sensor data remains unsolved. The potential of smart contracts to execute predefined actions based on sensor data is significantly diminished, limiting their practical applicability in IoT scenarios. Therefore, we consider developing oracle services for IoT-blockchain systems to solve the issue.

**Blockchain Oracle Services.** Existing studies primarily concern data authentication [17]–[20] and data validation [14], [16], [21]–[23]. The former ensures that data originates from authenticated sources and remains un-tampered. Existing studies implement authenticated oracle services that reliably transport un-tampered data from authenticated sources to smart contracts via the TEE [18], TLSNotary proof [24] or zero-knowledge proof [20]. The latter validates the accuracy of the data to prevent authenticated sources from providing Byzantine data to smart contracts. To reduce the risk of data manipulation and achieve data validation, most studies adopt decentralized model and propose voting mechanism [21], Schelling-point mechanism [14], and truth discovery mechanism [16]. Neither the voting mechanism nor the Schelling-point mechanism could deal with Byzantine data from authenticated but dishonest sources, only the truth discovery mechanism [16] can extract objects' truths from conflicting sources by performing truth inference. However, existing studies typically consider the static model, where the number of nodes is fixed and permanently active. In contrast, a dynamic model is more applicable to real network environments where nodes may join or leave arbitrarily. Thus, we account for designing dynamic decentralized oracle services in the partially synchronous network.

## VI. Conclusion

We develop new dynamic decentralized oracle services SenFEED, that enables smart contracts within IoT-blockchain applications to access accurate and real-time sensor data proactively. To address the challenges of implementing oracle services in IoT scenarios, we design the CompositeTD for accurate real-time truth inference on IoT streaming data and the DPS-BFT to handle liveness problems when service nodes join or leave the network. Experiments show that SenFEED can feed IoT data for smart contracts accurately and efficiently. Moreover, SenFEED supports large-scale service nodes with lower latency and higher throughput and supports the joining and leaving of nodes that do not affect the performance.

## Acknowledgment

This work was supported by the National Key R&D Program of China (No. 2021YFB2900100), the National Science Fund for Distinguished Young Scholars (No. 62025205), the National Natural Science Foundation of China (No. U22B2022, 62372383), and the Innovation and Technology Commission of Hong Kong (ITC) under Mainland-Hong Kong Joint Funding Scheme (No. MHP/135/23).

Authorized licensed use limited to: BEIJING UNIVERSITY OF POST AND TELECOM. Downloaded on January 26,2026 at 06:08:16 UTC from IEEE Xplore. Restrictions apply.

## REFERENCES


[1] L. Lao, Z. Li, S. Hou, B. Xiao, S. Guo, and Y. Yang, "A Survey of IoT Applications in Blockchain Systems: Architecture, Consensus, and Traffic Modeling," *ACM Computing Surveys*, vol. 53, no. 1, pp. 1–32, 2020.

[2] L. Hao, V. Naik, and H. Schulzrinne, "DBAC: Directory-Based Access Control for Geographically Distributed IoT Systems," in *Proc. of IEEE INFOCOM*, 2022.

[3] K. Fizza, A. Banerjee, P. P. Jayaraman, N. Auluck, R. Ranjan, K. Mitra, and D. Georgakopoulos, "A Survey on Evaluating the Quality of Autonomic Internet of Things Applications," *IEEE Communications Surveys & Tutorials*, vol. 25, no. 1, pp. 567–590, 2023.

[4] T. Cai, W. Chen, J. Zhang, and Z. Zheng, "SmartChain: A Dynamic and Self-Adaptive Sharding Framework for IoT Blockchain," *IEEE TSC*, vol. 17, no. 2, pp. 674–688, 2024.

[5] Y. Nan, X. Wang, L. Xing, X. Liao, R. Wu, J. Wu, Y. Zhang, and X. Wang, "Are You Spying on Me? Large-Scale Analysis on IoT Data Exposure through Companion Apps," in *Proc. of USENIX SECURITY*, 2023.

[6] M. S. Ali, M. Vecchio, M. Pincheira, K. Dolui, F. Antonelli, and M. H. Rehmani, "Applications of Blockchains in the Internet of Things: A Comprehensive Survey," *IEEE Communications Surveys & Tutorials*, vol. 21, no. 2, pp. 1676–1717, 2019.

[7] C. Profentzas, M. Almgren, and O. Landsiedel, "TinyEVM: Off-Chain Smart Contracts on Low-Power IoT Devices," in *Proc. of IEEE ICDCS*, 2020.

[8] F. Chen, J. Wang, C. Jiang, T. Xiang, and Y. Yang, "Blockchain Based Non-repudiable IoT Data Trading: Simpler, Faster, and Cheaper," in *Proc. of IEEE INFOCOM*, 2022.

[9] F. Daidone, B. Carminati, and E. Ferrari, "Blockchain-Based Privacy Enforcement in the IoT Domain," *IEEE TDSC*, vol. 19, no. 6, pp. 3887–3898, 2022.

[10] T. Rabehaja, S. Pal, A. Hill, and M. Hitchens, "A Blockchain-Based Approach for Parametric Insurance Under Multiple Sources of Truth," *IEEE TSC*, vol. 17, no. 3, pp. 718–732, 2024.

[11] "77+ Smart Contract Use Cases Enabled by Chainlink," https://blog.chain.link/smart-contract-use-cases/, [Accessed 27-06-2024].

[12] A. Pasdar, Y. C. Lee, and Z. Dong, "Connect API with Blockchain: A Survey on Blockchain Oracle Implementation," *ACM Computing Surveys*, vol. 55, no. 10, pp. 1–39, 2023.

[13] G. Wood, "Ethereum: A Secure Decentralised Generalised Transaction Ledger," *Ethereum project yellow paper*, 2014.

[14] L. Breidenbach, C. Cachin, B. Chan, A. Coventry, S. Ellis, A. Juels, F. Koushanfar, A. Miller, B. Magauran, D. Moroz *et al.*, "Chainlink 2.0: Next Steps in the Evolution of Decentralized Oracle Networks," *Chainlink Labs*, 2021.

[15] "UMA Protocol," https://docs.uma.xyz/, [Accessed 27-06-2024].

[16] Y. Xiao, N. Zhang, W. Lou, and Y. T. Hou, "A Decentralized Truth Discovery Approach to the Blockchain Oracle Problem," in *Proc. of IEEE INFOCOM*, 2023.

[17] S. Park, O. Bastani, and T. Kim, "ACon$^2$: Adaptive Conformal Consensus for Provable Blockchain Oracles," in *Proc. of USENIX SECURITY*, 2023.

[18] F. Zhang, E. Cecchetti, K. Croman, A. Juels, and E. Shi, "Town Crier: An Authenticated Data Feed for Smart Contracts," in *Proc. of ACM CCS*, 2016.

[19] J. Guarnizo and P. Szalachowski, "PDFS: Practical Data Feed Service for Smart Contracts," in *Proc. of ESORICS*, 2019.

[20] F. Zhang, D. Maram, H. Malvai, S. Goldfeder, and A. Juels, "DECO: Liberating Web Data Using Decentralized Oracles for TLS," in *Proc. of ACM CCS*, 2020.

[21] J. Adler, R. Berryhill, A. Veneris, Z. Poulos, N. Veira, and A. Kastania, "Astraea: A Decentralized Blockchain Oracle," in *IEEE iThings/GreenCom/CPSCom/SmartData*, 2018.

[22] J. Dong, C. Song, Y. Sun, and T. Zhang, "DAON: A Decentralized Autonomous Oracle Network to Provide Secure Data for Smart Contracts," *IEEE TIFS*, vol. 18, pp. 5920–5935, 2023.

[23] P. Chakka, S. Joshi, A. Kate, J. Tobkin, and D. Yang, "Oracle Agreement: From an Honest Super Majority to Simple Majority," in *Proc. of IEEE ICDCS*, 2023.

[24] "Provable Oraclize on Chainlink Ecosystem," https://www.chainlinkecosystem.com/ecosystem/provable-oraclize/, [Accessed 27-06-2024].

[25] A. Goknil, P. Nguyen, S. Sen, D. Politaki, H. Niavis, K. J. Pedersen, A. Suyuthi, A. Anand, and A. Ziegenbein, "A Systematic Review of Data Quality in CPS and IoT for Industry 4.0," *ACM Computing Surveys*, vol. 55, no. 14s, pp. 1–38, 2023.

[26] Q. Li, Y. Li, J. Gao, B. Zhao, W. Fan, and J. Han, "Resolving Conflicts in Heterogeneous Data by Truth Discovery and Source Reliability Estimation," in *Proc. of ACM SIGMOD*, 2014.

[27] P. S. Mukkamala, H. Wu, and B. Düdder, "Reliable and Streaming Truth Discovery in Blockchain-based Crowdsourcing," in *Proc. of IEEE SECON*, 2023.

[28] M. Castro and B. Liskov, "Practical Byzantine Fault Tolerance," in *Proc. of USENIX OSDI*, 1999.

[29] G. Zhang, F. Pan, Y. Mao, S. Tijanic, M. Dang'ana, S. Motepalli, S. Zhang, and H.-A. Jacobsen, "Reaching Consensus in the Byzantine Empire: A Comprehensive Review of BFT Consensus Algorithms," *ACM Computing Surveys*, vol. 56, no. 5, pp. 1–41, 2024.

[30] S. Duan and H. Zhang, "Foundations of Dynamic BFT," in *Proc. of IEEE S&P*, 2022.

[31] Z. Zhao, J. Cheng, and W. Ng, "Truth Discovery in Data Streams: A Single-Pass Probabilistic Approach," in *Proc. of ACM CIKM*, 2014.

[32] Y. Li, Q. Li, J. Gao, L. Su, B. Zhao, W. Fan, and J. Han, "On the Discovery of Evolving Truth," in *Proc. of ACM SIGKDD*, 2015.

[33] R. W. Ouyang, L. M. Kaplan, A. Toniolo, M. Srivastava, and T. J. Norman, "Parallel and Streaming Truth Discovery in Large-Scale Quantitative Crowdsourcing," *IEEE TPDS*, vol. 27, no. 10, pp. 2984–2997, 2016.

[34] A. Bessani, J. Sousa, and E. E. Alchieri, "State Machine Replication for the Masses with BFT-SMART," in *Proc. of IEEE/IFIP DSN*, 2014.

[35] "PBFT Python Implementation," https://github.com/kenzarh/PBFT, [Accessed 27-06-2024].

[36] A. Miller, Y. Xia, K. Croman, E. Shi, and D. Song, "The Honey Badger of BFT Protocols," in *Proc. of ACM CCS*, 2016.

[37] "Data Sets for Data Fusion Experiments," https://lunadong.com/fusionDataSets.htm, [Accessed 27-06-2024].

[38] "Intel Lab Data," https://db.csail.mit.edu/labdata/labdata.html, [Accessed 27-06-2024].

[39] X. Yin, J. Han, and P. S. Yu, "Truth Discovery with Multiple Conflicting Information Providers on the Web," *IEEE TKDE*, vol. 20, no. 6, pp. 796–808, 2008.

[40] W. Viriyasitavat, L. D. Xu, Z. Bi, D. Hoonsopon, and N. Charoenruk, "Managing QoS of Internet-of-Things Services Using Blockchain," *IEEE TCSS*, vol. 6, no. 6, pp. 1357–1368, 2019.

[41] Z. Wan, T. Zhang, W. Liu, M. Wang, and L. Zhu, "Decentralized Privacy-Preserving Fair Exchange Scheme for V2G Based on Blockchain," *IEEE TDSC*, vol. 19, no. 4, pp. 2442–2456, 2021.


Authorized licensed use limited to: BEIJING UNIVERSITY OF POST AND TELECOM. Downloaded on January 26,2026 at 06:08:16 UTC from IEEE Xplore. Restrictions apply.